

Thesis Predefense Prep — RLVR for Chart Question Answering

A topic-by-topic walkthrough of everything in your slide deck plus the standard committee follow-ups. Read this top-to-bottom once, then use the headers as a revision index.

1. LoRA (Low-Rank Adaptation)

Why it exists. Full fine-tuning of a 3B VLM means updating every weight matrix W in every transformer block. For Qwen2.5-VL-3B this is several tens of gigabytes of optimizer state (Adam keeps two moments per parameter, each fp32). On a single RTX 3090 with 24 GB VRAM, that is simply not possible — you cannot hold the model, gradients, optimizer state, activations, and a batch of chart images at the same time.

Core idea. Freeze the pretrained weights W_0 and learn a low-rank additive update:

$$\Delta W = B \cdot A \text{ where } A \in \mathbb{R}^{(r \times k)}, B \in \mathbb{R}^{(d \times r)}, \text{ and } r \ll \min(d, k)$$

The forward pass becomes $h = W_0 x + \Delta W \cdot x = W_0 x + B A x$. Only A and B receive gradients; W_0 stays frozen.

Why it works. Empirically, the change a fine-tune needs to apply to a pretrained weight matrix has a low "intrinsic rank" — you do not need full-rank updates to adapt a model to a narrow task. A rank of 8, 16, or 32 is typically enough.

Initialization. A is initialized with a Gaussian, B with zeros. So at step 0, $\Delta W = B A = 0$, meaning the model behaves exactly like the pretrained model. Training starts from a known-good point.

Parameter savings. For a weight matrix of shape 4096×4096 (≈ 16.8 M params), a rank-16 LoRA adds only $(4096 \cdot 16) + (16 \cdot 4096) \approx 131$ K params — a $\sim 128\times$ reduction for that layer. Across the whole model this is often $< 1\%$ of total parameters.

Other benefits worth mentioning if asked.

- **No inference latency.** At deployment time, you can merge BA back into W_0 , so inference cost is identical to the base model.
- **Modularity.** You can swap LoRA adapters in and out — one for chart QA, another for medical imaging, all sharing the same frozen backbone.
- **Stability.** Freezing W_0 protects the pretrained knowledge; catastrophic forgetting is reduced.

In your setup. You applied LoRA because the 3090's 24 GB VRAM forces it. This is exactly what the diagram on slide 17 shows — W_0 frozen, only the B/A paths carry backward gradients.

Likely follow-up: "Did you tune rank, alpha, target modules?" Have an answer ready — e.g., standard defaults are $r=16, \alpha=32$, applied to $q_proj, k_proj, v_proj, o_proj$ (attention) and sometimes MLP projections. Alpha acts as a scaling factor: the effective update is $(\alpha/r) \cdot BA$.

2. Why RL for Chart Reasoning — and Why Not SFT, DPO, or Others

Frame this as "every alternative has a specific failure mode that RLVR avoids."

2.1 Why not Supervised Fine-Tuning (SFT)

SFT maximizes the likelihood of a single ground-truth reasoning chain per (image, question) pair. Four problems:

1. **Expensive supervision.** Chain-of-thought rationales over charts have to be hand-authored or distilled from a larger model. Getting 30K+ high-quality CoT traces is costly and the quality ceiling is set by the teacher.
2. **Exposure bias.** SFT teacher-forces the correct token at every step; at inference the model has to condition on its own (possibly wrong) tokens. This mismatch hurts long-horizon reasoning in particular.
3. **Mode collapse / memorization.** SFT drives the policy toward one canonical reasoning chain. The model learns to mimic the *form* of reasoning in the training set rather than search for a correct path — Chart-RVR shows SFT producing the wrong answer while stating reasoning that is lexically fine (slide 11 case study, where SFT said "35" vs Chart-RVR's correct "25").
4. **No reward for correctness.** SFT rewards surface-level token matching, not whether the final answer is right. A rationale that is near-verbatim but arrives at a wrong number still gets a low loss.

2.2 Why not DPO (Direct Preference Optimization)

DPO trains on preference pairs (chosen y^+ , rejected y^-) using a simple classification loss derived from a Bradley-Terry model. It is elegant and stable, but:

1. **Needs preference pairs.** For chart QA, where does the "rejected" example come from? Unlike RLHF where humans rank two responses, here the natural signal is *correct* vs. *incorrect*, not preference between two correct answers. DPO on correct-vs-incorrect pairs is mathematically possible but wastes the rich verifiable-reward signal you already have.
2. **Single pair, single gradient direction.** DPO operates on pairs; it cannot naturally incorporate multi-component rewards (chart type + table + answer + process conformity) the way RL can. Everything collapses into one preference margin.
3. **No exploration.** DPO is off-policy on a fixed preference dataset; it cannot discover new reasoning paths during training. RL rollouts do.
4. **Temperature / diversity issues.** DPO has been shown to sharpen distributions aggressively (similar to PSR), which is the opposite of what you want for preserving reasoning diversity.

2.3 Why not RLHF with a learned reward model

Classic RLHF trains a reward model on human preferences then runs PPO against that reward.

1. **Reward hacking.** The reward model is a noisy proxy; the policy finds ways to get high reward without being genuinely better. For charts, a reward model might reward confident-looking reasoning that is factually wrong.
2. **Expensive.** Training a good reward model needs tens of thousands of human preference labels.
3. **Unnecessary.** You already have ground-truth answers, tables, and chart types. Why learn a model of correctness when correctness is directly checkable?

2.4 Why RL specifically for chart reasoning

- **Answers are verifiable.** Numeric answers can be compared with a tolerance; chart types are categorical; tables are structured. A rule-based verifier gives you dense, trustworthy signal.
- **Multiple rewards fuse naturally.** RL trivially handles $R_{\text{base}} + \lambda_1 \cdot \text{HCPC} + \lambda_2 \cdot \text{CLC}$. DPO and SFT cannot.
- **Exploration is valuable.** With $K=8$ rollouts per prompt, the model searches the space of reasoning chains and the reward tells it which paths matter. This is not available to SFT.
- **Generalization.** RL discovers a distribution of correct reasoning chains rather than one; this helps OOD (which your ChartFC results confirm).

A defensible one-liner: "SFT teaches the model to imitate reasoning; RLVR teaches it to *produce* reasoning that survives verification — these are fundamentally different training signals."

3. Reinforcement Learning — Crash Course

3.1 The formal setup

An RL problem is a Markov Decision Process: (S, A, P, R, γ) .

- **S:** set of states.
- **A:** set of actions.
- **$P(s'|s, a)$:** transition dynamics.
- **$R(s, a)$:** reward function.
- **$\gamma \in [0, 1)$:** discount factor.

The agent follows a **policy** $\pi(a|s)$ — a conditional distribution over actions given states. Starting from s_0 , it generates a trajectory $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$.

The **return** is $G_t = \sum_{k=0}^{\infty} \gamma^k \cdot r_{t+k}$. The objective is to find π that maximizes $J(\pi) = \mathbb{E}_{\tau \sim \pi} [G_0]$.

3.2 Value functions

- **State value:** $V^\pi(s) = \mathbb{E}[G_t \mid s_t = s, \pi]$
- **Action value:** $Q^\pi(s, a) = \mathbb{E}[G_t \mid s_t = s, a_t = a, \pi]$
- **Advantage:** $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ — how much better is action a than the average action at state s .

3.3 Two major families

1. **Value-based** (Q-learning, DQN): learn Q , act greedily. Works best in discrete action spaces.
2. **Policy-based** (REINFORCE, PPO, GRPO): directly optimize π_θ via gradient ascent on $J(\theta)$. Works well with continuous or very large action spaces — which is why LLMs use this family (the "action space" is the vocabulary at every token).

3.4 The policy gradient theorem (the heart of all of this)

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot A^\pi(s_t, a_t) \right]$$

In plain English: push up the log-probability of actions whose advantage is positive, push down those with negative advantage. This is the one equation the committee might ask you to write.

3.5 How LLMs map into this

For an LLM generating a response:

- **State s_t** = the prompt plus all previously generated tokens.
- **Action a_t** = the next token.
- **Policy π_θ** = the language model itself, softmax over vocabulary.
- **Reward** = typically only at the end of the sequence (sequence-level reward), e.g., whether the final answer matches ground truth.

This is called **sparse, trajectory-level reward**. Credit assignment (which token deserves blame or credit) becomes the central technical problem — and motivates everything that follows.

4. Evolution from Classical RL to RLVR

A timeline to have in your head:

1992 — REINFORCE (Williams). Pure policy gradient. High variance. Mostly toy.

2013 — DQN (Mnih et al.). Atari from pixels. Launched deep RL.

2015 — TRPO (Schulman et al.). Trust region: constrain each policy update so the new policy is "close" to the old in KL divergence. Stable but hard to implement.

2017 — PPO (Schulman et al.). Replaces TRPO's KL constraint with a clipped ratio. Much simpler, equally stable. Became the default for continuous control and, later, LLMs.

2017 — Christiano et al. introduce training from human preferences using learned reward models. This is the conceptual seed of RLHF.

2022 — InstructGPT / ChatGPT. OpenAI scales RLHF to language models. The recipe: SFT → reward model → PPO. Dominant paradigm until 2024.

2024 — DeepSeek introduces GRPO in DeepSeekMath. Drops the value function (critic) and normalizes rewards within a group of rollouts from the same prompt. Saves memory, makes RL feasible for much larger LLMs.

2024-2025 — RLVR era. DeepSeek-R1 and OpenAI o1 demonstrate that for *verifiable* domains (math, code, competitive programming), you can skip the reward model entirely and reward the policy purely based on whether its output is correct. Chain-of-thought reasoning emerges from this setup.

2025 — RLVR goes multimodal. Chart-RVR, BIGCHARTS-R1, Chart-R1, satellite-imagery RLVR (Köksal & Alatan). This is the neighborhood your thesis lives in.

2025 — RLVR decomposition. Zhu et al. (RLVR-Decomposed, NeurIPS 2025) show that negative reinforcement alone (NSR) often outperforms full RLVR at high Pass@k. This is the paper you are extending to VLMs.

The big shift RLVR represents. Reward model → rule-based verifier. Opaque learned scoring → transparent programmatic checking. This eliminates reward hacking at the cost of being limited to domains where answers can be checked. Chart QA fits perfectly.

5. Why RLVR for Chart QA Specifically

Four reasons, tuned for the committee:

1. **Charts have verifiable ground truth.** Unlike "write a nice essay," chart QA has a correct numeric answer, a correct chart type, and an underlying data table — all programmatically checkable.
2. **Rule-based rewards prevent hacking.** A numeric comparator with tolerance (relaxed correctness from ChartQA) cannot be fooled by confident prose. The policy must actually produce correct numbers.
3. **Multi-component rewards compose naturally.** R_base combines chart-type accuracy, table accuracy, and answer accuracy. HCPC adds cross-rollout consistency. CLC adds per-rollout grounding. This layering is what classical RLHF cannot do cleanly.
4. **Resource efficiency.** No reward model to train means you avoid a whole supervised pipeline. This is critical given your 3B-model, 1000-sample, single-GPU constraint.

The one-liner: "RLVR lets us replace human preference annotation with rule-based verification — which matches the structure of chart QA perfectly."

6. Policy Optimization Algorithms — the Zoo

6.1 REINFORCE

The simplest policy gradient. For each trajectory, apply gradient $\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot G_t$. High variance because G_t has no baseline.

6.2 Actor-Critic

Subtract a baseline $V^{\pi}(s_t)$ to get advantage $A_t = G_t - V^{\pi}(s_t)$, reducing variance. Requires learning V^{π} jointly — that's the "critic."

6.3 TRPO

Adds a KL constraint: $KL(\pi_{old} || \pi_{new}) \leq \delta$. Prevents destructive policy updates. Hard to implement (conjugate gradient, line search).

6.4 PPO (Proximal Policy Optimization) — the dominant baseline

PPO replaces TRPO's KL constraint with a **clipped surrogate objective**:

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta) \cdot \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \cdot \hat{A}_t)]$$

where $r_t(\theta) = \pi_{\theta}(a_t|s_t) / \pi_{\theta_{old}}(a_t|s_t)$ is the importance ratio and $\epsilon \approx 0.2$.

Intuition: you are allowed to move probability up for good actions and down for bad actions, but only so far. Beyond the clip, the gradient is zeroed out, so you cannot over-commit on one update.

PPO also requires a value function $V_{\phi}(s)$ (the critic) for advantage estimation, usually via GAE (generalized advantage estimation). For LLMs this means you carry a second model the same size as the policy — expensive.

6.5 GRPO (Group Relative Policy Optimization) — your main method

Motivation. For LLMs, the critic is a huge cost: you double memory and compute. GRPO eliminates the critic.

How. For a given prompt q , sample a **group** of G rollouts $\{o_1, \dots, o_G\}$ from the current policy. Compute the rule-based reward R_i for each rollout. Then the advantage for rollout i is simply:

$$\hat{A}_i = (R_i - \text{mean}(\{R_1, \dots, R_G\})) / \text{std}(\{R_1, \dots, R_G\})$$

This is a group-relative z-score. No critic needed — the group itself is the baseline.

The full objective mirrors PPO with this in-group advantage and an optional KL penalty against a reference model (usually the pre-RL model) to prevent excessive drift:

$$J_{\text{GRPO}}(\theta) = \mathbb{E} \left[(1/G) \sum_i (\min(r_i \cdot \hat{A}_i, \text{clip}(r_i, 1-\epsilon, 1+\epsilon) \cdot \hat{A}_i) - \beta \cdot \text{KL}(\pi_\theta \parallel \pi_{\text{ref}})) \right]$$

Why it matters for you. GRPO is what makes RL on a 3B VLM on a single 3090 feasible. No critic = roughly half the memory. The $K=8$ in your pipeline is the group size G .

Known weakness (slide 13). GRPO applies strong positive reinforcement to above-average rollouts, which sharpens the policy and collapses output diversity — hurting Pass@K at higher K .

6.6 NSR (Negative Sample Reinforcement) — your second method

From Zhu et al., RLVR-Decomposed (NeurIPS 2025). The key decomposition:

- **PSR (Positive Sample Reinforcement):** gradient step to *increase* $\pi_\theta(y|x)$ for correct y .
- **NSR (Negative Sample Reinforcement):** gradient step to *decrease* $\pi_\theta(y|x)$ for wrong y .

Standard RLVR does both. RLVR-Decomposed asks: what if we only do NSR?

$$\max_\theta -\pi_\theta(y|x) \text{ for wrong samples; no gradient for correct samples.}$$

Why this works surprisingly well. When you suppress a wrong output, probability mass does not go back to that one wrong sample — it redistributes across *all other plausible outputs*, many of which are correct. You get the benefit of learning which paths are wrong without forcing convergence onto a single correct path. Diversity is preserved implicitly.

What your thesis adds. NSR was tested only on text-math problems (MATH, AIME, AMC). You are the first to apply it to a VLM with multi-component rewards. That is a genuine contribution and the committee will press you on it — see §10.

6.7 W-REINFORCE (Weighted REINFORCE)

Also from Zhu et al. A compromise: keep both signals but weight them differently.

$$L_{\text{W-REINFORCE}} = \lambda \cdot L_{\text{PSR}} + L_{\text{NSR}}, \text{ with } \lambda \approx 0.1$$

Weak positive reinforcement, strong negative. Beats PPO and GRPO on MATH, AIME 2025, AMC23 in the original paper. You list it in your pipeline (slide 16) as a method you investigate though your primary results report GRPO, GRPO+HCPC, and NSR.

7. Pass@k Metrics

7.1 Definition

Given a prompt, sample n rollouts from the model and count c correct ones. The probability that k uniformly-sampled rollouts contain at least one correct is:

$$\text{Pass}@k = 1 - C(n-c, k) / C(n, k)$$

Pass@1 is the expected accuracy of a single sample. Pass@4 is the probability that at least one of four samples is correct.

7.2 What Pass@k really measures

- **Pass@1** → quality of the top-probability mass. Reflects "average" accuracy.
- **Pass@k for large k** → *reasoning capacity* or *diversity*. If Pass@k grows fast with k , the model has many distinct attempts that sometimes succeed. If Pass@k plateaus, the model has collapsed to a few modes.

This matters for your thesis because **positive reinforcement sharpens the policy and raises Pass@1 but collapses Pass@k at large k** (slide 12, RLVR-Decomposed figure). NSR preserves high Pass@k.

7.3 CoT-Pass@k (Wen et al., slide 9)

Pass@k only checks the final answer. A model can guess correctly through flawed reasoning — you cannot tell. CoT-Pass@k requires *both* correct reasoning *and* correct final answer. Wen et al. prove that RLVR improves CoT-Pass@k at all k , meaning RLVR improves reasoning quality (not just sampling efficiency). This is the theoretical backing for why you care about *path-consistent* rewards like HCPC, not just answer accuracy.

7.4 Why you report Pass@1, @2, @4

- Pass@1 = headline accuracy.
- Pass@2, Pass@4 = show whether gains come from mode collapse (Pass@1 up, Pass@4 flat) or genuine diversity-preserving improvement (Pass@k grows throughout).

Your results show exactly this distinction — see §9.

8. The Pipeline — Step by Step, with Justification

8.1 Seven-step flow (slide 15)

1. **Input.** Chart image I , question Q , ground truth (type, table, answer).
2. **Generation.** Sample $K = 8$ rollouts from the current policy π_θ (with temperature > 0 for diversity).
3. **Parsing.** Each rollout is expected to produce `<type>`, `<table>`, `<think>`, `<answer>` tags. Extract these components.
4. **Filtering.** Identify the **Fully Correct (FC)** subset: rollouts where chart type matches GT, final answer matches GT (with relaxed correctness for numerics), and table accuracy ≥ 0.8 . FC is the input to HCPC.
5. **Rewards.** Compute per-rollout $R_{\text{total}} = R_{\text{base}} + \lambda_1 \cdot R_{\text{HCPC}} + \lambda_2 \cdot R_{\text{CLC}}$.
6. **Advantages.** Method-dependent: GRPO does group-relative z-score; NSR zeros gradient on correct rollouts; W-REINFORCE reweights.
7. **Policy update.** Apply PPO-style clipped gradient.

8.2 Why this decomposition makes sense

Why $K = 8$ rollouts. You need enough diversity in the group to compute meaningful advantages (for GRPO) and a non-empty FC subset most of the time (for HCPC). $K=8$ is a common choice — large enough for statistical signal, small enough for memory. Smaller K (e.g., 2 or 4) gives noisy advantages and often empty FC sets on hard prompts.

Why structured output (`<type><table><think><answer>`). This is the key engineering move. It makes each component of the response independently parseable, which makes each component independently rewardable. Without structured output, you cannot compute table accuracy, type accuracy, and answer accuracy separately. The system prompt (slide 18) enforces this format.

Why R_{base} . Baseline accuracy reward: did the model get chart type, table, and final answer right? This is the core verifiable reward — the "RLVR" part.

Why R_{HCPC} (your contribution). Consider two failure modes:

- *Extraction drift:* a rollout gets the right answer but misread the table (lucky guess).
- *Reasoning collapse:* multiple rollouts all use the identical reasoning chain, suggesting the model has memorized a single path.

R_{HCPC} is defined over the FC subset with two scores:

- **Table Consistency (x):** among FC rollouts, do they all have accurate table data? Rewards extraction consistency — prevents lucky guesses.
- **Reasoning Diversity (y):** among FC rollouts, is the reasoning textually distinct? Rewards exploration — prevents mode collapse.
- $\text{HCPC Score} = w_2 \cdot x + w_3 \cdot y$.

Key design choice: HCPC is **conditional on correctness**. You reward diverse reasoning *only* among correct rollouts. This is "correct-path consistency" — you are not rewarding diversity for its own sake (which could encourage random variation), but diversity that arrives at correct answers.

Why R_CLC. Per-rollout grounding check: every referenced numeric value must appear in the extracted table, no hallucinated numbers, logical coherence preserved across steps. This is the "no hallucination" constraint — it directly addresses the objective "eliminate hallucinated reasoning by grounding every inference step in chart-extracted values" (slide 3).

Why combine all three. R_base gives the *correctness* signal. R_HCPC gives the *structural quality* signal. R_CLC gives the *faithfulness* signal. None of them alone is sufficient:

- R_base alone → Chart-RVR / vanilla GRPO → mode collapse, hallucinated reasoning that happens to land on correct numbers.
- R_HCPC alone → rewards diversity without correctness anchor.
- R_CLC alone → grounded reasoning that might still be wrong.

The sum — $R_{\text{base}} + \lambda_1 \cdot \text{HCPC} + \lambda_2 \cdot \text{CLC}$ — is a hierarchical signal: correctness first, then consistency across rollouts, then grounding within rollouts.

8.3 Justification of iterating GRPO vs NSR vs W-REINFORCE

Each policy-update method handles the reward signal differently:

- **GRPO:** strong PSR + NSR → best at exploiting correctness, worst at preserving diversity.
- **NSR-only:** no PSR → weaker exploitation, stronger implicit diversity, expected to help OOD.
- **W-REINFORCE:** weighted middle ground.

Testing all three lets you ask the central research question (slide 14): "*Is explicit diversity promotion (HCPC) competitive with implicit diversity preservation (NSR)?*"

9. Results — What the Numbers Actually Say

9.1 In-Distribution (ChartQA + PlotQA, 500 samples)

Method	Pass@1	Pass@2	Pass@4
Base (no training)	59.45	74.33	84.46
GRPO	73.00	82.20	87.31
GRPO+HCPC (yours)	72.85	84.57	88.70
NSR (yours, first VLM)	73.40	83.72	88.00

Observations.

- All RL methods beat the base by a wide margin: +13.55 to +13.95 at Pass@1. This answers RQ1: yes, RLVR with 1000 samples and a 3B VLM gives meaningful gains.
- NSR takes Pass@1; GRPO+HCPC takes Pass@2 and Pass@4. The pattern is consistent with theory: explicit diversity engineering (HCPC) maximizes diversity-sensitive metrics; negative-only reinforcement (NSR) gives a sharper Pass@1.
- GRPO and GRPO+HCPC differ by only 0.15 at Pass@1 but 2.37 at Pass@2 and 1.39 at Pass@4. This is exactly what you'd expect if HCPC is preventing mode collapse — the first sample is roughly as accurate, but the model has more genuinely distinct correct attempts.

9.2 Out-of-Distribution (ChartFC, 500 samples)

Method	Pass@1	Pass@2	Pass@4
GRPO	62.45	79.90	81.80
GRPO+HCPC	61.22	80.10	83.14
NSR	68.70	82.60	92.40

This is the headline finding. NSR dominates OOD by a striking margin: +6.25 Pass@1 over GRPO, +10.60 Pass@4 over GRPO. GRPO+HCPC is actually worse than GRPO at Pass@1 on OOD.

Why NSR wins OOD. The mechanism is the central claim of the RLVR-Decomposed paper, now demonstrated on VLMs: positive reinforcement sharpens the policy onto training-distribution-specific reasoning patterns. When the distribution shifts (ChartQA/PlotQA → fact-checking), those sharpened patterns transfer poorly. NSR never sharpens — it only suppresses wrong answers — so the implicit diversity carried over from the base model is preserved.

Why HCPC doesn't help OOD. HCPC's reasoning-diversity score is measured textually on *training-distribution FC* sets. The diversity it rewards is diversity in the kinds of reasoning that appear during chart-QA training. On ChartFC (fact-checking), the required reasoning structure is different (binary verification rather than numeric extraction), so HCPC's training-time diversity reward does not translate into OOD robustness.

The Pass@4 OOD gap (81.8 → 92.4) is the most important single number in the deck. It is direct evidence that diversity preservation is the mechanism driving generalization — exactly the RQ4 answer.

9.3 What each experiment answers

- **E0 → E1 (Base → GRPO):** RQ1 answered. Yes, RLVR with 1K samples on a 3B VLM works.
- **E1 → E2 (GRPO → GRPO+HCPC):** RQ2 partially answered. Reasoning-path-aware reward improves diversity metrics ID; does not improve OOD.
- **E1 → E3 (GRPO → NSR):** RQ3 answered. NSR transfers from text-math to VLMs with multi-component rewards. It is not only compatible, it is *better* on OOD.
- **E2 vs E3:** RQ4 answered. Implicit diversity preservation (NSR) beats explicit diversity engineering (HCPC) on OOD. This is the non-obvious finding.

9.4 What the results do *not* show (be honest about this)

- **No statistical significance tests.** You trained once per method; the 0.15 pp gaps are within run-to-run variance for RL. Expect the committee to ask.
 - **Single base model.** Qwen2.5-VL-3B only. Gains could be architecture-specific.
 - **ChartFC is one OOD distribution.** Generalization claims would be stronger with more OOD benchmarks (ChartBench, EvoChart, ChartQAPro).
 - **1000 samples may be too small to fully saturate any method.** The rankings could shift at 10K or 34K samples.
 - **No direct SFT comparison.** You cannot definitively claim RL > SFT in your own results — only cite prior work for that.
-

10. Conclusions and Likely Follow-up Questions

10.1 Your core contributions

1. **HCPC.** A hierarchical, correctness-conditioned reward that combines table consistency (across correct rollouts) and reasoning diversity (among correct rollouts). Improves ID Pass@2/Pass@4 over vanilla GRPO.
2. **NSR for VLMs.** First application of negative-only reinforcement to a vision-language model with multi-component rewards. Delivers dominant OOD performance — the most important empirical finding.
3. **Side-by-side comparison.** Explicit (HCPC) vs implicit (NSR) diversity under identical backbone, data, and verifier. This is the comparison no prior work has run.

10.2 Questions the committee will likely ask — with short answers

"Why only 1000 samples?" Compute limit (single 3090) and a deliberate RQ1 framing: can RLVR work under resource constraints? The answer is yes. Scaling is future work.

"Why K=8?" Memory and statistical trade-off. Fewer than 8 gives noisy group baselines and often empty FC sets; more than 8 exceeds VRAM at 3B with LoRA plus activations.

"How do you compute table accuracy?" Cell-wise match with relaxed numeric tolerance and permutation-invariant row matching. The 0.8 threshold is standard in ChartQA evaluation.

"How do you measure reasoning diversity in HCPC?" Textual distinctness among FC rollouts — likely n-gram overlap, BLEU divergence, or edit distance between reasoning chains. Be ready with the exact implementation.

"Why does GRPO+HCPC underperform GRPO on OOD Pass@1?" HCPC's reasoning-diversity reward is trained on chart-QA distribution; it may reinforce diversity patterns that do not transfer to fact-checking. The result suggests *explicit* diversity is distribution-specific, while *implicit* diversity (NSR) is not.

"What is the difference between HCPC and CLC?" HCPC is **across rollouts** (does the FC group agree on the table? is reasoning distinct?). CLC is **within a rollout** (are all referenced values in the table? any hallucinated numbers? logically coherent?). Both are reported in the pipeline; CLC is listed as future work to integrate fully (slide 22).

"Why didn't you combine HCPC and NSR?" Acknowledged as future work (slide 22, "Implement CLC and NSR-HCPC Pipelines"). The current pairwise comparison isolates their individual contributions; combination is next.

"Statistical significance?" Acknowledge single-run results, note RL run-to-run variance, point to the OOD Pass@4 gap (+10.6 points) as a gap too large to be noise.

"What about reward hacking?" R_base is rule-based (exact-match with tolerance) → not hackable. R_HCPC is conditioned on correctness → cannot reward wrong-answer diversity. R_CLC is a grounding check → explicitly penalizes hallucinated numbers. The design is structurally hack-resistant.

"What is a rollout?" One completed generation from the policy — a full (type, table, think, answer) response sampled end-to-end. With $K=8$, you get 8 such rollouts per training prompt.

"What does 'advantage' mean in your setup?" For GRPO, the z-scored reward of a rollout within its group. For NSR, effectively the reward minus an implicit zero — no gradient on correct, negative gradient on incorrect. The advantage determines the *sign and magnitude* of the policy update for each rollout.

"Why is PPO not in your table?" Memory: PPO needs a critic network of roughly the policy size. With LoRA fine-tuning on a 3B VLM on a 3090, carrying a critic is not feasible. GRPO was built exactly to avoid this.

"Why Qwen2.5-VL and not Llava or InternVL?" Qwen2.5-VL-3B is the current best-performing open 3B-class VLM on chart-style visual tasks, has robust tool support (transformers, vLLM), and is the backbone used in Chart-RVR (base paper, slide 11), giving a clean comparison line.

"Why is the base model score so low (59.45)?" Base Qwen2.5-VL-3B was not trained for structured `<type><table><think><answer>` output; it sometimes fails to follow the format, which the parser then cannot score. RL rapidly teaches format adherence in the first few hundred steps — a large chunk of the +13 Pass@1 gain is from format learning, not reasoning learning.

"Then is the reasoning gain actually smaller than 13 points?" Yes, and this is worth acknowledging. A format-controlled baseline (prompt-engineered to produce the tags) would give a fairer zero-training number. This is honest future work.

10.3 Limitations to state upfront (defuses hostile questions)

- 3B scale only; 7B/72B results may differ.
- 1K training samples; full 34K will change the numbers.
- One OOD benchmark (ChartFC); broader OOD is future work.
- Single-run results without significance testing.
- HCPC's textual diversity measure is a surface-level proxy; entropy regularization over rollout distributions (slide 22, future work) would be principled.

10.4 The two-sentence summary to have ready

"We show that RLVR with only 1,000 samples can meaningfully improve a 3B VLM's chart QA ability, and we propose HCPC — a correctness-conditioned reward that enforces cross-rollout extraction consistency and rewards reasoning diversity. Our strongest result is that NSR, a negative-only reinforcement scheme originally proposed for text math, transfers to multimodal chart reasoning and dominates all other methods on out-of-distribution chart fact-checking, suggesting that implicit diversity preservation generalizes better than explicit diversity engineering."

That is the thesis. Everything else is scaffolding.

11. Quick Revision Cheat-Sheet

- **LoRA:** $W = W_0 + BA$, rank $r \ll d$. Freeze W_0 , train A and B. Fits 3B on 24 GB.
- **RL:** maximize $\mathbb{E}[\sum_t \gamma^t r_t]$ under π_θ . Policy gradient: $\nabla \log \pi \cdot A$.
- **PPO:** clipped ratio objective, needs critic.
- **GRPO:** z-score reward within group of K rollouts, no critic.
- **NSR:** gradient only on wrong rollouts. Preserves diversity.
- **RLVR:** rule-based reward, no reward model.
- **Pass@k:** $1 - C(n-c, k)/C(n, k)$. Measures diversity at high k.
- **HCPC:** cross-rollout reward over FC set — table consistency + reasoning diversity.
- **CLC:** per-rollout grounding check — values in table, no hallucinations.
- **R_total** = $R_{\text{base}} + \lambda_1 \cdot \text{HCPC} + \lambda_2 \cdot \text{CLC}$.
- **Headline result:** NSR Pass@4 on OOD ChartFC = 92.4 vs GRPO 81.8.

Good luck.